

PROJECT_QLE

Libya Petroleum Exploration & Interpretation Platform

Complete Technical Documentation

Version 1.0 | Generated May 17, 2026 | Libya NOC

Confidential — Authorised Users Only

Table of Contents

- 1. Project Overview & Architecture**
 - 1.1 What is Project_QLE?
 - 1.2 System Architecture Diagram
 - 1.3 Directory Structure
 - 1.4 Technology Stack
- 2. Core Data Models (core/models.py)**
 - 2.1 Why Dataclasses?
 - 2.2 Enumeration Classes
 - 2.3 WellHeader, WellCurve, WellLog
 - 2.4 Seismic, Zone, Reservoir, Report models
- 3. Libya Geology Configuration (core/libya_geology.py)**
 - 3.1 Basin Parameter Design
 - 3.2 All Parameters Explained
 - 3.3 Known Fields Database
- 4. File Parsers (parsers/)**
 - 4.1 LAS Parser (las_parser.py)
 - 4.2 File Parser (file_parser.py)
 - 4.3 SEG-Y Parser (segy_parser.py)
- 5. Petrophysical Analysis (analysis/petrophysics.py)**
 - 5.1 Vshale Algorithms
 - 5.2 Porosity Equations
 - 5.3 Water Saturation Equations
 - 5.4 Pore Pressure — Eaton Method
 - 5.5 Permeability Correlations
 - 5.6 PetrophysicsEngine Class
- 6. Facies Classification (analysis/facies.py)**
 - 6.1 Rule-Based Classifier
 - 6.2 KMeans Unsupervised
 - 6.3 Random Forest Supervised
 - 6.4 Neural Network (MLP)
- 7. Reservoir Characterisation (analysis/reservoir.py)**
 - 7.1 Cut-off Logic
 - 7.2 Net Pay Calculation
 - 7.3 Fluid Contact Detection
 - 7.4 Volumetrics — STOIP & GIIP
 - 7.5 FZI and Lorenz Coefficient
- 8. Statistical Analysis (analysis/statistics.py)**
 - 8.1 Descriptive Statistics
 - 8.2 Normality Tests
 - 8.3 Outlier Detection
 - 8.4 Monte Carlo Uncertainty
- 9. Log Correlation (analysis/log_correlation.py)**
 - 9.1 Depth Resampling

[9.2 Pearson + Cross-Correlation](#)

[9.3 Formation Top Picking](#)

10. Petrophysical Summaries ([analysis/petro_summaries.py](#))

[10.1 Porosity / Permeability / Saturation Summaries](#)

[10.2 Formation Descriptions](#)

[10.3 DST Interpretation](#)

11. Machine Learning ([ml/](#))

[11.1 ModelComparer — LR / RF / XGBoost](#)

[11.2 TrendAnalyzer — Depth Trends](#)

[11.3 Metrics: MAE, RMSE, R-squared](#)

[11.4 Model Serialisation](#)

12. AI Interpretation ([ai/gemini_interpreter.py](#))

[12.1 Gemini API Integration](#)

[12.2 Libya-Specific System Prompt](#)

[12.3 Auto Model Detection](#)

13. Database Layer ([database/](#))

[13.1 SQLite Schema Design](#)

[13.2 ORM Models \(\[db.py\]\(#\)\)](#)

[13.3 Access Control \(\[auth.py\]\(#\)\)](#)

[13.4 Key Generation & Hashing](#)

14. Pipeline Orchestrator ([pipeline.py](#))

[14.1 QLEPipeline Design](#)

[14.2 The 5-Step Workflow](#)

15. Streamlit Application ([app.py](#))

[15.1 Page Architecture](#)

[15.2 Access Control Wall](#)

[15.3 Core Plot Functions](#)

[15.4 Side-by-Side Log Comparison](#)

[15.5 All 18 Pages Explained](#)

16. Error Reference & Known Issues

17. Deployment & Setup Guide

1. Project Overview & Architecture

What we built and why

1.1 What is Project_QLE?

Project_QLE is a full-stack petroleum exploration interpretation platform designed specifically for Libyan petroleum geology. It was built to replace fragmented, manual workflows (spreadsheets, isolated scripts, paper logs) with a unified system that: reads raw LAS and SEG-Y well files, performs industry-standard petrophysical calculations, classifies lithological facies using both deterministic rules and machine learning, trains and compares ML regression models (Linear Regression, Random Forest, XGBoost) for property prediction, generates professional reservoir summaries, and provides Gemini AI-powered geological narratives calibrated to Libyan basin conventions.

The platform is built on Python + Streamlit and runs entirely in a browser. All data is persisted in a local SQLite database. Access is controlled via SHA-256-hashed access keys issued by the project owner — no unauthorised user can open the application without a valid key.

1.2 System Architecture Diagram

Layer	Component	Responsibility
UI	app.py (Streamlit)	All 18 interactive pages, plots, forms
Orchestration	pipeline.py	Chains parsers → analysis → AI in sequence
Analysis	analysis/	Petrophysics, Facies, Reservoir, Stats, Correlation
ML	ml/	Model training, comparison, trend analysis
AI	ai/	Gemini API, geological prompts, map generation
Parsers	parsers/	LAS, SEG-Y, PDF, CSV, XML, DOCX, JPG
Core	core/	Shared dataclasses, Libya geology config
Database	database/	SQLite via SQLAlchemy, user auth
Config	core/libya_geology.py	All basin petrophysical parameters

1.3 Directory Structure

Every directory is a Python package (has an `__init__.py`). This allows clean relative imports throughout the project.

```
Project_QLE/
├── app.py           ← Streamlit UI (18 pages, ~1800 lines)
├── pipeline.py      ← QLEPipeline – chains all steps
├── __init__.py      ← Exports QLEPipeline
├── core/
│   ├── models.py    ← All shared dataclasses
│   ├── libya_geology.py ← Basin defaults, field database
│   └── parsers/
└── database/
```

■ ■■■■	las_parser.py	← LAS 1.2 / 2.0 / 3.0 via lasio
■ ■■■■	file_parser.py	← PDF, DOCX, XML, CSV, JPG
■ ■■■■	segy_parser.py	← SEG-Y seismic via segyio
■■■■	analysis/	
■ ■■■■	petrophysics.py	← Vshale, Porosity, Sw, Pore Pressure, Perm
■ ■■■■	facies.py	← Rule-based, KMeans, RF, MLP classifiers
■ ■■■■	reservoir.py	← Net pay, OWC, STOIP, FZI, Lorenz
■ ■■■■	statistics.py	← Descriptive, normality, outliers, MC
■ ■■■■	log_correlation.py	← Cross-well Pearson + DTW
■ ■■■■	petro_summaries.py	← Summaries, formation descriptions, DST
■■■■	ml/	
■ ■■■■	model_comparison.py	← LR / RF / XGBoost training + comparison
■ ■■■■	trend_analysis.py	← Depth-trend LR + XGBoost
■■■■	ai/	
■ ■■■■	gemini_interpreter.py	← Gemini API with Libya prompts
■ ■■■■	map_generator.py	← Interpolated property maps
■■■■	database/	
■ ■■■■	db.py	← SQLAlchemy ORM models
■ ■■■■	auth.py	← User management, key hashing
■■■■	tests/	
■ ■■■■	test_core.py	← Synthetic Libya well unit tests

1.4 Technology Stack

Streamlit	Web UI framework. Renders the browser-based dashboard with widgets, plots, file uploaders.
lasio	Industry-standard LAS file parser. Reads LAS 1.2, 2.0, 3.0 well log formats.
segyio	SEG-Y seismic file parser for reading trace headers and sample data.
NumPy	Array operations for all petrophysical calculations — vectorised, fast, NaN-safe.
Pandas	DataFrame storage for well data; used throughout for filtering, grouping, export.
SciPy	Statistical tests (Shapiro-Wilk, K-S, linear regression, cross-correlation).
scikit-learn	KMeans clustering, Random Forest, Linear Regression, StandardScaler, metrics.
XGBoost	Gradient boosting trees for ML comparison and trend analysis.
Matplotlib	All log track plots, crossplots, histograms, comparison figures.
SQLAlchemy	ORM layer over SQLite — defines all database tables as Python classes.
ReportLab	Used to generate this PDF documentation from Python code.
google-generativeai	Google Gemini AI API SDK — geological narrative generation.
hashlib/secrets	SHA-256 key hashing and cryptographic key generation for access control.

2. Core Data Models

core/models.py

2.1 Why Dataclasses Instead of Pydantic?

The original prototype used Pydantic BaseModel, which adds runtime type validation and serialisation. However, Pydantic v2 changed its API significantly and was not always available in the target environment (GitHub Codespaces). We migrated to Python's built-in @dataclass decorator, which requires no external dependency, works identically across Python 3.8+, and still provides clear type hints. The trade-off is that we lose automatic validation — for example, a float field can receive a string without raising an error — but for a geoscience data pipeline where inputs are already validated at parse time, this is acceptable.

2.2 Enumeration Classes

We use Python Enum (specifically str Enum, so each value IS its string label) for four concepts: FileType, CurveType, Facies, and FluidType. Using Enum instead of raw strings prevents typos — if a developer writes Facies.SNDSTONE Python raises AttributeError immediately at development time rather than a silent wrong label at runtime. The str Enum inheritance means Facies.SANDSTONE == 'Sandstone' is True, so these values can be stored directly in databases and DataFrames.

FileType	PDF, DOCX, XML, JPG, CSV, LAS, SEGY, UNKNOWN — tells parsers which reader to dispatch
CurveType	GR, SP, RT, RHOB, NPHI, DT, CALI, PE, DEPT, MD, TVD, OTHER — standard log mnemonics
Facies	Sandstone, Shale, Limestone, Dolomite, Coal, Anhydrite, Salt, Unknown — lithological classes
FluidType	Oil, Gas, Water, Dry, Unknown — reservoir fluid classification

2.3 WellHeader, WellCurve, WellLog

WellHeader stores the metadata from the LAS file WELL section: well name, UWI (Unique Well Identifier — an industry standard 16-character identifier), field name, company, coordinates, Kelly Bushing elevation, total depth, start/stop depths, and the null value marker (conventionally -999.25 in LAS files — this number was chosen historically because it is unlikely to occur as a real measurement).

WellCurve represents one log track (e.g. GR, RHOB). It stores the mnemonic (the short curve name), unit, description text, CurveType classification, and the data as a Python list of floats. The .array property converts this list to a NumPy array on demand — we store as a list because Python lists serialise cleanly to JSON/SQLite, whereas NumPy arrays do not.

WellLog is the top-level container. It holds one WellHeader, a dictionary of WellCurve objects (keyed by mnemonic string), an optional Pandas DataFrame for convenient column-wise access after petrophysics, and the source file path. get_depth() tries DEPT, MD, DEPTH in that order — this handles the common inconsistency between LAS producers who use different depth column names.

2.4 Interpretation Result Models

ZoneInterval	One depth interval with geological interpretation: top/base depths, facies, fluid, porosity, Sw, permeability, pore pressure, and AI confidence score.
ReservoirSummary	Well-level aggregate: net pay thickness, average porosity/Sw/permeability, fluid contact depth (OWC or GOC), AI narrative text.
CorrelationResult	Stores the Pearson r correlation coefficient and depth lag (in metres) between two wells for one curve.
StatisticalResult	Full descriptive statistics for one curve: N, mean, std, min, max, P10/P50/P90, skewness, kurtosis, histogram bin data.
InterpretationReport	Top-level output from QLEPipeline: aggregates all wells, reservoirs, correlations, statistics, and the AI executive summary.
SeismicTrace	One SEG-Y trace: trace number, inline/crossline coordinates, sample list, sample rate, delay time.
SeismicDataset	Collection of SeismicTrace objects plus metadata from the binary and text headers.

3. Libya Geology Configuration

core/libya_geology.py

3.1 Why a Dedicated Libya Geology File?

Petrophysical calculations are not universal — every basin has different rock physics. For example, the Sirte Basin has Paleocene carbonate reservoirs with a matrix density of 2.71 g/cc (limestone), while the Ghadames and Murzuq basins have Paleozoic sandstone reservoirs with matrix density 2.65 g/cc (quartz). Using the wrong matrix density shifts every porosity calculation by several percentage points, which can change a 'Good' reservoir to a 'Poor' one. By centralising all basin defaults in one file, any developer can update Sirte parameters in one place and every calculation automatically picks up the change.

3.2 All Parameters Explained

gr_clean	Gamma Ray reading in pure, clay-free rock (GAPI). Sirte carbonates are very clean — GR=12 GAPI. Used as the lower end of Vshale normalisation.
gr_shale	Gamma Ray reading in 100% shale (GAPI). Used as the upper end of Vshale normalisation. Shales in Sirte read ~90 GAPI.
rho_matrix	Matrix density of the dominant rock type (g/cc). Limestone = 2.71, Sandstone = 2.65, Dolomite = 2.87. Critical for density porosity calculation.
rho_fluid	Formation fluid density (g/cc). Fresh mud filtrate = 1.00, slightly saline = 1.05, salt mud = 1.10. Affects density porosity directly.
rw	Formation water resistivity (ohm-m). Lower Rw = saltier water. Sirte Paleocene has Rw~0.025 (very saline). Critical for Archie Sw.
rsh	Shale resistivity (ohm-m). Used in Simandoux Sw equation for shaly sands. Governs how much shale conductivity 'short-circuits' the true Sw.
vsh_method	Which Gamma Ray to Vshale transform to use. Iarionov_young for Mesozoic/older rocks (Sirte). Iarionov_old for Tertiary (gives higher Vsh). linear is conservative.
overburden_gradient	Rate of overburden pressure increase with depth (psi/ft). Drives pore pressure calculation via Eaton method. Varies 0.92-1.00 psi/ft across Libya.
hydrostatic_gradient	Rate of hydrostatic pressure increase (psi/ft). 0.433 = fresh water, 0.465 = seawater. Offshore uses 0.465. This is the 'normal' pressure baseline.
normal_dt_surface	Sonic travel time at surface for Eaton normal compaction trend (microsec/ft). Higher = softer surface sediments. Offshore values are higher.
normal_dt_exp	Exponential decay constant for normal compaction. Negative value causes dt to decrease with depth (rock compacts and gets faster). Typical: -0.00020 to -0.00025.
typical_api	Informational string — the typical API gravity of produced oil. Used by Gemini AI in its narrative reports. Not used in calculations.
typical_gor	Informational string — the typical Gas-Oil Ratio in the basin. Used by Gemini AI. Not used in calculations.

3.3 Known Libyan Fields Database

LIBYAN_FIELDS is a list of dictionaries containing 12 major Libyan fields with their basin, dominant fluid type, API gravity, and GPS coordinates. The coordinates allow the Home page to render a map of field locations using Streamlit's `st.map()`. The database is intentionally not exhaustive — it covers the fields most likely to be referenced in a Sirte, Ghadames, or Murzuq exploration programme.

4. File Parsers

[parsers/las_parser.py](#) | [file_parser.py](#) | [segy_parser.py](#)

4.1 LAS Parser ([las_parser.py](#))

LAS (Log ASCII Standard) is the universal file format for well logs. It was defined by the Canadian Well Logging Society and has three versions: 1.2, 2.0, and 3.0. We use the lasio library to handle the parsing complexity (section detection, header extraction, null value substitution). Our wrapper adds the following on top:

- Null value replacement: LAS uses -999.25 to mean 'no data'. We convert these to NumPy NaN so that all downstream calculations are NaN-safe automatically.
- Mnemonic normalisation: LAS files from different vendors use different names for the same curve (RHOB vs RHOZ, DT vs DTC vs DTCO). Our `_MNEMONIC_MAP` converts all variants to a canonical CurveType.
- DataFrame construction: We build a Pandas DataFrame from the raw curves for convenient columnar access during petrophysics. Column names are uppercased for consistency.
- Header extraction: `_extract_header()` reads the WELL section using a safe helper that returns "" rather than raising KeyError if a field is absent — LAS files in the field are often incomplete.

4.2 File Parser ([file_parser.py](#))

A dispatch table (`_PARSERS` dict) maps each FileType enum to a parser function. `parse_file()` detects the file type from its extension, looks up the parser, calls it, and returns a unified `ParsedFile` dataclass regardless of input format. This design means the rest of the system only deals with `ParsedFile` objects — the specific file format is an implementation detail.

<code>_parse_pdf</code>	Uses PyMuPDF (fitz). Iterates pages, extracts text with <code>page.get_text()</code> , extracts embedded images via <code>extract_image()</code> . Returns raw text joined with newlines.
<code>_parse_docx</code>	Uses python-docx. Extracts paragraphs and all table cells. Multi-table documents: only the first table becomes the primary DataFrame; all tables stored in extra dict.
<code>_parse_xml</code>	Uses lxml etree. Parses XML tree, attempts to flatten first-level children into a tabular DataFrame (one row per child, columns from sub-element tags).
<code>_parse_image</code>	Uses Pillow. Reads image metadata (format, mode, size), computes mean RGB values via NumPy array statistics, stores raw bytes for downstream use.
<code>_parse_csv</code>	Uses Pandas <code>read_csv</code> with <code>pd.to_numeric(errors='ignore')</code> to auto-convert numeric-looking string columns. Returns shape, column names, dtype map in metadata.

4.3 SEG-Y Parser ([segy_parser.py](#))

SEG-Y is the standard format for seismic data. `segio.open()` is used with `ignore_geometry=True` for post-stack / 2D data where trace headers may be sparse. `f.mmap()` memory-maps the file for fast access without loading all traces into RAM. The `max_traces` parameter allows reading only the first N traces from a large survey for preview purposes — full surveys can have hundreds of thousands of traces.

5. Petrophysical Analysis

analysis/petrophysics.py

5.1 Vshale Algorithms

Vshale (volume of shale) is the fraction of each depth sample that consists of clay/shale. It is the most fundamental derived curve — it controls how much of the porosity is 'useless' clay-bound water. All methods start from the Gamma Ray Index (IGR) which linearly normalises GR between the clean baseline and shale value.

Linear (IGR)	$V_{sh} = IGR$. Simplest and most conservative. Overestimates V_{sh} in older rocks. Used as a quick quality check.
Larionov Young	$V_{sh} = 0.083 * (2^{(3.7*IGR)} - 1)$. Calibrated for Mesozoic and older rocks. Standard choice for Sirte carbonates and Paleozoic sandstones. Gives lower V_{sh} than linear at high IGR values.
Larionov Old	$V_{sh} = 0.33 * (2^{(2*IGR)} - 1)$. Calibrated for Tertiary (young) rocks. Ghadames uses this because Acacus sands have Tertiary-style GR responses.
Clavier	$V_{sh} = 1.7 - \sqrt{3.38 - (IGR + 0.7)^2}$. Empirical equation from North Sea data. Used occasionally for cross-check.
Stieber	$V_{sh} = IGR / (3 - 2*IGR)$. Gives intermediate values. Alternative empirical method.

Note: The clamp() function clips all V_{sh} results to $[0, 1]$ after calculation. Without clamping, mathematical overshoot near the GR baseline or shale line produces physically impossible negative or >1.0 values.

5.2 Porosity Equations

Three independent porosity measurements are computed when the required curves exist. The best available is used as PHIE (effective porosity):

Density Porosity (PHID)	$PHID = (\rho_{matrix} - \rho_{HOB}) / (\rho_{matrix} - \rho_{fluid})$. The density log measures the bulk density of the formation. Knowing the end-member densities (pure matrix and pure fluid) allows solving for porosity. This is the most reliable single porosity measurement in consolidated formations.
N-D Crossplot (PHIND)	$PHIND = \sqrt{((NPHI^2 + PHID^2) / 2)}$. Combining neutron (NPHI) and density (PHID) porosities as a root-mean-square average cancels out some lithology effects and is particularly sensitive to gas-bearing intervals (gas cross-over effect where $PHID > NPHI$).
Sonic Wyllie (PHIS)	$PHIS = ((DT - DT_{matrix}) / (DT_{fluid} - DT_{matrix})) / CP$. The Wyllie time-average equation assumes the formation is a simple mixture of matrix and fluid acoustic paths. CP is a compaction correction factor (1.0 = no correction needed, typically for well-consolidated formations above 2000m).
Sonic Raymer (PHIS_raymer)	$PHIS = 0.625 * (1 - DT_{matrix} / DT)$. An improved formula for consolidated formations that does not assume a simple time-average mixture. Gives more accurate results at low porosity ($<10\%$).

Priority: PHIND is preferred (uses two independent tools for better accuracy), then PHID (single density tool), then PHIS (sonic is most affected by borehole conditions). If no density or neutron log exists, PHIS is used.

5.3 Water Saturation Equations

Water saturation (S_w) is the fraction of pore space filled with water. $1 - S_w$ = hydrocarbon saturation. The resistivity log (RT) is the primary input — hydrocarbons are non-conductive (high resistivity) while saline water is conductive (low resistivity).

Archie (clean sands)	$S_w^n = (a * R_w) / (RT * \phi^m)$. Parameters: a =tortuosity factor (1.0), m =cementation exponent (2.0, higher = tighter rock), n =saturation exponent (2.0). Valid only in clean sands with <15% clay. The cementation exponent m encodes how tortuous the fluid pathways are — higher m means pores are more disconnected and resistivity rises faster with decreasing S_w .
Simandoux (shaly sands)	Accounts for the additional conductivity path through clay minerals. The clay term $d=V_{sh}/R_{sh}$ adds a parallel conduction path that lowers resistivity independently of water saturation. Without this correction, Archie overestimates S_w (under-estimates hydrocarbon saturation) in shaly formations. Used whenever $V_{SHALE} > 15\%$.

5.4 Pore Pressure — Eaton's Method

$PP = OBG - (OBG - HG) * (dt_{normal} / dt_{obs})^n$ where OBG =overburden pressure, HG =hydrostatic pressure, n =Eaton exponent (3.0). The key insight is that in a normally compacted formation, sonic travel time (DT) decreases predictably with depth as rock compacts. If DT is anomalously high (slower than expected) at a given depth, the rock is under-compacted, which means pore pressure is above hydrostatic — an overpressure hazard. The normal compaction trend is modelled as $dt_{normal} = A * \exp(-B * depth_{ft})$ where A and B are calibrated per basin. All results are in psi.

5.5 Permeability Correlations

Timur	$k = 0.136 * (\phi^{4.4}) / (S_{wi}^2)$. Empirically derived from clean sandstone core data. The $\phi^{4.4}$ relationship means small changes in porosity cause large changes in permeability (nonlinear). S_{wi} is the irreducible water saturation — the water that cannot be produced even at infinite displacement pressure. Used for Ghadames and Murzuq sandstones.
Carmen-Kozeny	$k = C * (\phi^3 / (1-\phi)^2) * (1/SSA^2)$. Surface area per unit volume (SSA) dominates — fine-grained rocks have enormous SSA and tiny k . Used for Sirte carbonates where grain size distribution matters more.
Coates (NMR)	Approximates the NMR permeability transform: $k = (\phi/a)^b * ((1-S_{wi})/S_{wi})^c * 1000$. The ratio $(1-S_{wi})/S_{wi}$ is the free-fluid ratio — the fraction of pore fluid that can actually flow vs. capillary-bound fluid. Used when NMR-style interpretation is desired.

5.6 PetrophysicsEngine Class

PetrophysicsEngine is the high-level coordinator that runs the full workflow on a WellLog object. Its `__init__` resolves all parameters by checking: (1) explicit keyword arguments passed by the user, (2) the Libya basin defaults from `get_basin_defaults()`, (3) hard-coded fallback values. This three-level priority system means a user can override a single parameter (e.g. `rw=0.05`) without having to specify all other basin parameters. The `run()` method returns an enriched Pandas DataFrame with all derived columns appended. The original `well.df` is updated in-place so subsequent pages in the Streamlit app can access the derived curves.

6. Facies Classification

[analysis/facies.py](#)

6.1 Why Multiple Classification Methods?

Facies classification assigns a rock type label (Sandstone, Shale, Limestone, etc.) to each depth sample. No single method is universally best: rule-based methods are transparent and fast but require expert parameter tuning; KMeans finds natural groupings in the data without labels but the cluster-to-facies mapping is arbitrary; Random Forest and MLP require labelled training data but can recognise complex multi-curve patterns. The platform provides all four so the geologist can choose the most appropriate for their data availability and time constraints.

6.2 Rule-Based Classifier

Uses three logs: GR (clay content), RHOB (density), and NPHI (neutron porosity). Decision tree: if $GR \geq gr_shale_min$ → SHALE. Else if $GR \leq gr_sand_max$ and $RHOB \geq rho_lim_evap$ → ANHYDRITE (very dense, GR-clean rock = evaporite). Else if $RHOB \geq rho_lim_carb$ → DOLOMITE or LIMESTONE depending on density. Else → SANDSTONE. The thresholds are adjustable via constructor parameters so the geologist can calibrate to their specific basin log responses.

6.3 KMeans Unsupervised Classifier

StandardScaler normalises each input curve to zero mean and unit variance before KMeans clustering. This prevents curves with large numerical ranges (e.g. RT in ohm-m can range 1–1000) from dominating curves with small ranges (VSHALE 0–1). `n_clusters` defaults to 5 matching the 5 common lithologies. After clustering, cluster integers are mapped to Facies enum values. Known limitation: the cluster-to-facies mapping is hardcoded (cluster 0 → Sandstone, etc.) which may not match the actual cluster content — a proper implementation would inspect cluster centroids and sort by GR value.

6.4 labels_to_zones()

After classification, consecutive depth samples with the same facies label are merged into ZoneInterval objects. The function scans the array once, tracking the current facies label and the top depth. When the label changes, a zone is closed and a new one opened. Zones thinner than `min_thickness_m` (default 0.5m) are discarded — this prevents noise spikes from creating single-sample 'zones'.

7. Reservoir Characterisation

[analysis/reservoir.py](#)

7.1 Cut-off Logic (CutoffSet)

Net reservoir is defined as rock that meets ALL four criteria simultaneously: PHIE $\geq$ phi_min (minimum porosity to flow), SW $\leq$ sw_max (must contain hydrocarbons), VSHALE $\leq$ vsh_max (not too clayey to flow), PERM_mD $\geq$ perm_min (minimum flow capacity). These thresholds differ by basin — Sirte carbonates are productive at lower porosities (phi_min=0.06) because fractures add flow capacity not captured in matrix porosity. Ghadames/Murzuq clastics need phi_min=0.08 because they lack fracture enhancement.

7.2 Net Pay Calculation

apply_cutoffs() returns a boolean mask. compute_net_pay() applies np.gradient() to the depth array to get the thickness of each sample interval, then sums the thicknesses where the mask is True. Using gradient rather than a fixed step size handles irregularly-sampled LAS files (where the depth step may vary due to logging speed changes). Result is in metres.

7.3 Fluid Contact Detection (OWC / GOC)

detect_fluid_contact() scans downward through the Sw curve looking for the depth where Sw crosses sw_threshold (default 0.5). At this transition, the function linearly interpolates between adjacent samples to find the precise crossing depth rather than returning the nearest sample depth. This gives sub-sample resolution at the cost of slight interpolation error. The detected depth is the OWC (Oil-Water Contact) or GOC (Gas-Oil Contact) depending on the saturation context.

7.4 Volumetrics — STOIP and GIIP

STOIP (Stock-Tank Oil Initially In Place) in stock-tank barrels: $STOIP = 7758 * area_acres * net_pay_ft * phi * (1-Sw) / Bo$. The constant 7758 converts acre-feet to barrels. Bo (oil formation volume factor) converts reservoir barrels to surface barrels (oil shrinks as gas comes out of solution at surface conditions, so $Bo > 1.0$).

GIIP (Gas Initially In Place) in MSCF: $GIIP = 43560 * area_acres * net_pay_ft * phi * (1-Sw) / (Bg * 1000)$. The constant 43560 converts acres to square feet. Bg (gas formation volume factor) converts reservoir cubic feet to surface cubic feet (gas expands enormously at surface pressure, so $Bg \ll 1.0$). These are deterministic single-point estimates — Monte Carlo uncertainty is handled separately in the Statistics page.

7.5 FZI and Lorenz Coefficient

Flow Zone Indicator ($FZI = \sqrt{k/phi} / (phi/(1-phi))$) quantifies reservoir quality per unit pore volume. Samples with similar FZI belong to the same hydraulic flow unit — they will behave similarly during production regardless of depth. This is the Amaefule et al. 1993 method, standard in Libya NOC reporting.

Lorenz Coefficient (0=homogeneous, 1=extreme heterogeneity) measures how unevenly flow capacity ($k \cdot h$) is distributed. Computed as: sort samples by k/phi descending (best flow units first), compute cumulative $k \cdot h$ and cumulative pore volume, plot one against the other (Lorenz curve), measure the area above the 45-degree line. A Lorenz coefficient above 0.7 signals extreme layering that will cause early water breakthrough and poor sweep efficiency.

8. Statistical Analysis

[analysis/statistics.py](#)

8.1 Descriptive Statistics

`descriptive_stats()` computes N (non-NaN count), mean, std, min, max, P10/P50/P90 percentiles, skewness, and kurtosis for a single log curve. It also computes a 50-bin histogram (counts and bin edges) for visual display. P10/P90 are particularly important in petroleum engineering: P10 is the optimistic estimate (only 10% of samples are below this value), P90 is the pessimistic estimate. P50 is the median — it is preferred over the mean for skewed distributions like permeability, which has a log-normal distribution.

8.2 Normality Tests

Three normality tests are implemented: Shapiro-Wilk (most powerful for small samples, limited to 5000 samples), Kolmogorov-Smirnov (compares empirical CDF to theoretical normal), and D'Agostino-Pearson (tests for excess skewness and kurtosis). A p-value > 0.05 conventionally means we cannot reject the null hypothesis that the data is normally distributed. This matters because many standard statistical methods assume normality — if GR is not normally distributed, a simple mean may be misleading.

8.3 Outlier Detection

Three methods: IQR fence (values beyond $Q1 - 1.5 \cdot IQR$ or $Q3 + 1.5 \cdot IQR$), Z-score (values more than 3 standard deviations from the mean), and Isolation Forest (multivariate — detects samples that are anomalous across multiple curves simultaneously). In well logging, outliers often indicate tool malfunctions (spike noise), washed-out borehole sections (caliper > bit size → spurious RHOB), or genuine geological anomalies (thin hard streaks). Distinguishing these requires geological context, which is why the Isolation Forest result is displayed for the geologist to interpret rather than automatically removed.

8.4 Monte Carlo Porosity Uncertainty

`monte_carlo_porosity()` samples N values from a normal distribution defined by the user's mean and standard deviation, clips to [0,1] (physically meaningful range), and reports P10/P50/P90 of the resulting distribution. This quantifies uncertainty in STOIP/GIIP calculations: instead of one deterministic porosity value, the geologist enters their best estimate and uncertainty range, and the system shows the resulting range of reservoir size estimates.

9. Log Correlation

analysis/log_correlation.py

9.1 Depth Resampling

Two wells almost never have the same depth sampling interval or depth range. `resample_to_common_depth()` finds the overlapping depth range, computes a common step size (median of both wells' depth increments), creates a regular grid over the overlap, and interpolates both curves onto this grid using `np.interp()` (linear interpolation). Only the overlapping depth range is used for correlation — extrapolation outside the data range is never performed.

9.2 Pearson Correlation + Cross-Correlation

Pearson r measures linear correlation (-1 =perfect inverse, 0 =none, $+1$ =perfect). It captures whether log patterns move together across wells — high r on GR suggests the same shale intervals are present in both wells at the same depth, supporting stratigraphic continuity.

Cross-correlation (`np.correlate` with `mode='full'`) scans all possible depth shifts between `-max_lag` and `+max_lag` samples. The shift that maximises the correlation is the depth lag — how much one well needs to be shifted to align with the other. A positive lag means Well A's patterns arrive earlier (shallower) than Well B's, which can indicate structural dip between the wells.

9.3 Formation Top Auto-Picking

`pick_formation_tops()` finds the N depths with the highest GR values. High GR typically marks shale layers, which are the most consistent correlatable markers between wells. These are crude initial picks — the geologist is expected to review and refine them using the Formation Tops page. The auto-picks serve as a starting point to accelerate the manual picking workflow.

10. Petrophysical Summaries

analysis/petro_summaries.py

10.1 Porosity, Permeability, and Saturation Summaries

Each summary function computes the full statistical profile (mean, std, min, max, P10/P50/P90) plus a quality classification: Poor/Fair/Good/Excellent. The quality thresholds are:

Porosity	Poor <5%, Fair 5-10%, Good 10-18%, Excellent >18% (PHIE). Carbonate carbonates can be excellent at 15% due to fractures; sandstone needs >18% to be excellent.
Permeability	Poor <1 mD, Fair 1-10 mD, Good 10-100 mD, Excellent >100 mD. One millidarcy (mD) allows ~1 barrel/day/ft under 1 psi/ft pressure gradient.
Saturation	Fluid type inferred from Sw: Sw<30% → Oil, Sw 30-50% → Gas, Sw>70% → Water. Mixed fluid interpreted as transition zone or dual-phase reservoir.

The permeability geometric mean (log-space average) is computed because permeability is log-normally distributed. The arithmetic mean is strongly influenced by a few very high-k streaks that may not represent the bulk flow behaviour. The geometric mean better represents the typical sample in a log-normal distribution and is used in most industry reservoir simulators.

10.2 Formation Descriptions and Lithology Text

LITHO_DESCRIPTIONS is a dictionary mapping lithology names to standardised geological descriptions. When a formation top is added, describe_formation() retrieves the appropriate text and appends measured average porosity and permeability if available from the petrophysics run. This auto-generates the lithology description paragraph that would normally be written manually by the geologist for each formation.

10.3 DST Interpretation

interpret_dst() formats all available DST parameters into a readable block. The skin factor interpretation is built-in: skin > 5 means the formation is damaged (drilling mud invasion, clay swelling, fines migration) and stimulation (acid job) is recommended. Skin < -2 means the well is artificially stimulated (fractures or acid fracturing). Skin near 0 means an undamaged, unfractures formation. This interpretation is displayed alongside the raw numbers in the DST Tests page.

11. Machine Learning

ml/model_comparison.py |
ml/trend_analysis.py

11.1 Why These Three Algorithms?

Linear Regression is the baseline — if the relationship between logs and the target property is linear, LR will capture it with minimal overfitting. It is interpretable (the coefficient for each input feature shows its relative importance and direction).

Random Forest builds many decision trees on random subsets of data and features (bootstrap aggregation = 'bagging'), then averages their predictions. This reduces overfitting compared to a single tree. It handles non-linear relationships and interactions between features naturally, and the feature_importances_ attribute shows which log curves were most predictive.

XGBoost (Extreme Gradient Boosting) builds trees sequentially where each tree corrects the errors of the previous ones. It is generally the highest-accuracy algorithm for tabular data but is more prone to overfitting if not tuned. We compare all three so the geologist can see whether the extra complexity of XGBoost is justified by a meaningful improvement in R-squared.

11.2 Model Evaluation Metrics

MAE (Mean Absolute Error)	Average absolute difference between predicted and actual values. In the same units as the target (e.g. if predicting PHIE, MAE is in fraction units: 0.02 = 2% average error). Robust to outliers — one large error does not dominate the metric.
RMSE (Root Mean Squared Error)	Like MAE but squares errors before averaging, then takes the square root. RMSE penalises large errors more heavily than MAE. If RMSE >> MAE, the model makes occasional very large errors that need investigation.
R-squared (R2)	Fraction of variance in the target explained by the model. 1.0 = perfect prediction, 0.0 = model no better than predicting the mean, negative = model worse than predicting the mean. R2 > 0.8 is generally considered a good predictive model for well log data.

11.3 Data Split — Why 80/20?

The data is split 80% training, 20% testing (configurable). The test set is kept completely separate during training and used only to evaluate final performance. This prevents data leakage — if we trained and tested on the same data, the model would appear far more accurate than it actually is on new wells. Note: for depth-series data (log curves), we split sequentially (first 80% of depth samples for training, last 20% for testing) rather than randomly, because random splitting would give the model information from shallow AND deep samples during training, making the test set unrealistically easy.

11.4 TrendAnalyzer — Depth Trends

TrendAnalyzer fits models where depth is the ONLY input feature. This answers a different question than ModelComparer: 'Does this property systematically increase or decrease with depth?' rather than 'Can I predict this property from other log curves?'. The linear regression gives a slope coefficient in units-per-metre. XGBoost captures non-linear depth trends (e.g. porosity may compact linearly to 3000m then compact more rapidly below that). The resulting predicted curves are plotted alongside the actual log for visual validation.

11.5 Model Serialisation

`serialize_model()` pickles the trained model object to bytes, then base64-encodes the bytes to a plain ASCII string. This string can be stored in the SQLite database TEXT column without binary encoding issues. `deserialize_model()` reverses the process. Security note: pickle is not safe to load from untrusted sources — only models trained within the platform should be loaded.

12. AI Interpretation

ai/gemini_interpreter.py

12.1 Why Gemini and Not Another Model?

Google Gemini was chosen over other LLMs for several reasons: (1) Free API tier is available via Google AI Studio, making the platform accessible without a paid subscription. (2) Gemini 1.5 Pro has a 1 million token context window, allowing large amounts of petrophysical data to be included in the prompt without truncation. (3) The API is accessible via the google-generativeai Python SDK, which has a simpler interface than some alternatives. The system is designed to be AI-agnostic — the GeminiInterpreter class could be replaced with an OpenAI or Anthropic wrapper by changing only the `_call()` method.

12.2 Libya-Specific System Prompt

The system prompt (the invisible instruction that preconditions the AI's behaviour) defines the AI as a 30-year petroleum geology expert specialising in Libyan basins. It instructs the model to: reference Libyan formation names (Intisar, Sarir, Acacus, Mamuniyat, etc.), apply North African structural context (Tethyan margin, inversion tectonics), use NOC reporting conventions, apply specific reservoir quality thresholds calibrated to Libyan data, and always end with an exploration risk rating. This specificity dramatically improves the relevance of the generated text compared to a generic geology prompt.

12.3 Auto Model Detection

The original implementation hardcoded 'gemini-1.5-flash' which caused a 404 error because Google changed their model naming convention (adding 'models/' prefix). `_auto_select_model()` calls `genai.list_models()` to get the live list of available models, then tries each candidate in priority order: `models/gemini-1.5-pro` first (best quality), then `models/gemini-1.5-flash` (faster), then `models/gemini-pro` (stable free tier fallback). This makes the system robust to future API changes.

13. Database Layer

[database/db.py](#) | [database/auth.py](#)

13.1 Why SQLite?

SQLite stores the entire database in a single file (`~/project_qle/database.db`). This requires no database server installation, no port configuration, and no network access — the file travels with the project. For a single-instance exploration platform used by one team, SQLite's limitations (no concurrent writes from multiple processes) are not relevant. SQLAlchemy provides an ORM layer so the code is database-agnostic: migrating to PostgreSQL would require changing only the connection string.

13.2 ORM Schema — All Tables

projects	One row per exploration project. Fields: name (unique), basin, description, created_at, updated_at. Parent of all other tables via foreign keys.
wells	One row per LAS file loaded and saved. Fields: project_id (FK), well_name, uwi, field_name, coordinates, depths, las_file_path, uploaded_at.
petro_data	One row per depth sample per well. Fields: well_id (FK), depth_m, and all petrophysical computed values (vshale, phie, sw, sh, perm_md, pore_pressure_psi, facies). This table can be large — a 3000m well at 0.1m sampling = 30,000 rows.
formation_tops	One row per formation top per well. Fields: well_id (FK), formation_name, depth_m, lithology, description, confidence, picked_at.
dst_tests	One row per DST test. Fields: well_id (FK), all DST parameters (pressures, flow rates, fluid type, permeability, skin, GOR, API, temperature).
interpretations	One row per saved interpretation run. Stores JSON summaries of porosity/permeability/saturation statistics and the AI narrative text.
ml_models	One row per trained ML model. Stores model name, target variable, input features, performance metrics (MAE, RMSE, R2), training sample count, and the serialised model bytes.
users	One row per authorised user. Fields: username (unique), key_hash (SHA-256), role (owner/user), is_active, created_at, last_seen, notes.

13.3 Access Control System

On first launch, `init_auth()` checks whether an owner account exists. If not, it generates a random 24-character access key using Python's `secrets` module (cryptographically secure random number generator), hashes it with SHA-256, creates the owner user record, and writes the plaintext key to `~/project_qle/owner_key.txt`. The key file is never overwritten on subsequent launches — the owner must retrieve the key from this file.

Users are created by the owner via the admin dashboard (`admin_dashboard.py`). Each user gets a unique generated key (format: `QLE-xxxxxxxxxxxxxxxxxxxx`). The plaintext key is shown ONCE at creation time and never stored — only the SHA-256 hash is persisted. If a user loses their key, the owner can regenerate a new one (old key immediately invalid) via the admin dashboard.

13.4 Why SHA-256?

SHA-256 is a one-way cryptographic hash function. Given a key, computing its hash is fast. Given a hash, recovering the original key is computationally infeasible. This means that even if the database is stolen, the attacker cannot recover valid access keys — they see only hashes. `authenticate()` hashes the presented key and compares it to the stored hash, never comparing plaintext to plaintext.

Note: For higher security in production, consider PBKDF2 or bcrypt which add a 'salt' and iteration count to make brute-force attacks computationally expensive. For the current use case (internal team tool, short random keys), SHA-256 is adequate.

14. Pipeline Orchestrator

pipeline.py

14.1 QLEPipeline Design

QLEPipeline uses a fluent builder pattern: `pipe.add_las('file.las').add_las('file2.las')` chains method calls because each `add_las()` returns `self`. This makes pipeline construction readable in scripts. The pipeline also accepts already-parsed `WellLog` objects via `add_well()` — used by the Streamlit app which parses files in the upload page and passes the objects directly.

14.2 The 5-Step Workflow

Step 1: Petrophysics	<code>PetrophysicsEngine.run()</code> on every well. Updates <code>well.df</code> with derived columns. Failures are caught, logged, and added to <code>report.warnings</code> rather than crashing the whole pipeline.
Step 2: Facies + Reservoir	Classifies facies (KMeans default), merges into zones via <code>labels_to_zones()</code> , builds <code>ReservoirSummary</code> with net pay, average properties, and OWC depth. Each well's summary is appended to <code>report.reservoirs</code> .
Step 3: Statistics	<code>batch_stats()</code> computes descriptive statistics for standard log curves (GR, RHOB, NPHI, RT, PHIE, SW, PERM_mD) for every well. Results go into <code>report.statistics</code> .
Step 4: Cross-well Correlation	<code>correlate_well_suite()</code> runs pairwise correlation for all well pairs on GR and RHOB. Only runs if ≥ 2 wells are loaded. Results in <code>report.correlations</code> .
Step 5: AI Interpretation	<code>GeminiInterpreter.interpret_reservoir()</code> generates a narrative for each reservoir summary. <code>summarise_report()</code> writes the project-level executive summary. Only runs if <code>use_ai=True</code> and a valid Gemini key is available.

All steps use `try/except` to catch and record errors without aborting subsequent steps. This is essential for field data which is often incomplete — one well with a missing GR curve should not prevent the other five wells from being processed.

15. Streamlit Application

app.py (~1800 lines)

15.1 Page Architecture

app.py is structured as a single file with a top section of shared code (constants, helper functions, shared plot functions) followed by one large if/elif block — one branch per page. Streamlit re-runs the entire script on every user interaction, so all state is stored in `st.session_state` (a dictionary persisted across reruns). The `ss()` helper initialises keys with defaults on first access to prevent `KeyError`.

15.2 Access Control Wall

`_check_access()` checks whether 'authenticated_user' key exists in `session_state`. If not, `_login_wall()` renders a key entry form and calls `st.stop()` which halts all further script execution — no page content is rendered until the key is validated. On successful authentication, the user's username and role are stored in `session_state` and the app reruns to show the full interface. The owner role gets a crown emoji (👑) and access to admin features like deleting projects. Regular users can only read and add data.

15.3 Core Plot Functions

<code>make_log_plot()</code>	Multi-track well log plot. Creates 1×N subplots sharing the y (depth) axis. Each curve gets a track coloured according to <code>TRACK_COLORS</code> . Log-scale tracks (RT, PERM_mD) use <code>ax.semilogx()</code> . Zone overlay is drawn as <code>ax.axhspan()</code> colour bands with 12% opacity so the curve is still visible. OWC line is <code>ax.axhline()</code> in blue.
<code>make_comparison_plot()</code>	Side-by-side comparison. Creates 1×(2N) subplots all sharing one y-axis (<code>sharey=True</code>). Left N tracks = Well A, right N tracks = Well B. A subtle separator marker is drawn on the last Well A track. Both wells share the same depth axis so the same formation depths align horizontally. This was the key bug fix — the old implementation used <code>sharey='row'</code> which stacked the two wells vertically instead.
<code>make_facies_track()</code>	1.2cm-wide single track. For each consecutive depth pair, <code>ax.fill_betweenx()</code> fills the depth interval with the facies colour. This renders as a colour-coded column showing facies variation with depth.
<code>make_crossplot()</code>	2D scatter plot with optional colour coding by a third variable (default <code>VSHALE</code>). A Pearson r line or contours would be a useful future addition.
<code>make_histogram()</code>	Single-curve frequency histogram with P10/P50/P90 lines overlaid as dashed vertical lines.

15.4 The 18 Application Pages

■ Home

Shows loaded well count, reservoir zone count, basin name, AI status. Renders the Libya field map using `st.map()` with GPS coordinates from `LIBYAN_FIELDS`. Shows the 6-step workflow diagram as metric cards.

■ Data Upload

st.file_uploader() for LAS files (multi-file). Each file is written to a temp file, parsed via parse_las(), the temp file is always deleted in a finally block (prevents disk leaks). Parsed WellLog objects stored in session_state['wells'].

■ Project Manager

Lists all projects from the SQLite database. Each project shows its saved wells and allows saving the currently-loaded wells to the project. Owners can delete projects.

■ Well Log Viewer

Selectbox for well, multiselect for curves, two number_input widgets for depth window (Top/Base). The depth inputs replace the old slider for more precise control. Zone overlay and OWC line toggled by checkboxes. If FACIES column exists, a narrow facies track is rendered below the log plot.

■ Log Comparison

Selects two wells and curves. Computes the overlapping depth range. Two number_inputs for shared Top/Base zoom. Calls make_comparison_plot() for the TRUE side-by-side layout. Optional Pearson correlation table below the plot.

■ Petrophysics

Configuration column: well select, basin select, advanced parameter expander. Run button triggers PetrophysicsEngine with the configured parameters and stores the result in session_state['petro_{well_name}']. Results shown in three tabs: derived log tracks (with depth zoom), N-D crossplot + porosity-perm crossplot, raw data table with CSV download.

■ Petro Summary

Generates porosity/permeability/saturation summaries using petro_summaries.py. Shows quality badges (Excellent/Good/Fair/Poor). Renders porosity histogram, log-permeability histogram, saturation pie chart. Full text summary paragraph auto-generated by build_bundle(). Additional petrophysical crossplots (GR vs RHOB, Porosity vs Sw).

■ Facies Analysis

Selectbox for classification method (KMeans or Rule-Based) and cluster count. After classification, shows pie chart of facies proportions, facies track, and log plot with zone overlay, all scoped to the depth zoom window.

■ Statistics

Four tabs: Descriptive (P10/P50/P90 table), Histograms (curve selector), Correlation Matrix (heatmap with values), Monte Carlo (porosity uncertainty simulation with configurable mean/std/N).

■ Log Correlation

Runs correlate_well_suite() for all well pairs. Shows Pearson r, depth lag, and quality rating table. Auto-picked formation tops table. GR overlay plot (all wells on one depth axis for visual correlation).

■ Reservoir Summary

Builds ReservoirSummary for all wells. Table with net pay, average phi/Sw/k, OWC. Reservoir log view with zone overlay and OWC line. Volumetric inputs (area, Bo, Bg) for STOIP/GIIP calculation. Zone detail table.

■ Formation Tops

Two tabs: Pick Tops (form to enter formation name, top/base depth, lithology, notes) and Table/Descriptions (all picked tops in a table plus auto-generated lithology description for each formation). Tops are rendered as horizontal lines on the log plot with name labels.

■ DST Tests

Form to enter all DST parameters. interpret_dst() formats the results as a readable text block with skin interpretation. Summary table of all tests for the selected well.

■ ML Comparison

ModelComparer trains LR, RF, and XGBoost on selected features and target. Shows performance comparison table and bar charts. Random Forest feature importance bar chart. Predict-at-custom-values tool — enter feature values and get predictions from all models simultaneously.

■ Trend Analysis

TrendAnalyzer fits LR and XGBoost models with depth as the only input. Shows model comparison table, linear trend equation and direction, and a side-by-side plot: actual log (left) vs. model predictions overlaid on actual (right).

■ Map View

Field Map tab: filters LIBYAN_FIELDS by basin, renders with st.map(). Property Map tab: requires ≥ 2 wells with GPS coordinates; interpolates reservoir properties using scipy griddata and renders as a contour map.

■ AI Interpretation

Three tabs: Reservoir Narrative (Gemini interprets one well's reservoir summary), Correlation Commentary (Gemini analyses cross-well correlation results), Geological Q&A; (free-text question with optional reservoir data context injected into the prompt).

■ Full Report

Project name input ABOVE the button (critical — Streamlit widget-inside-button bug fix). Runs QLEPipeline with all loaded wells. Displays AI executive summary, per-well reservoir summaries with AI narratives, warnings list, and CSV export button.

16. Error Reference & Known Issues

Bugs fixed and known limitations

Bug: Gemini 404 model not found

Root cause: Google changed their model naming from 'gemini-1.5-flash' to 'models/gemini-1.5-flash'. Fix: `_auto_select_model()` calls `genai.list_models()` to discover available models dynamically rather than hardcoding the name. Falls back through a priority list of candidate names.

Bug: `AttributeError: 'InterpretationReport' has no attribute 'basin'`

Root cause: The `InterpretationReport` dataclass was defined without a `basin` field, but `app.py` accessed `report.basin` on the Full Report page. Fix: Added `basin` field to `InterpretationReport` with default 'SIRTE'. App uses `getattr(report, 'basin', basin)` as an additional safety net.

Bug: Petrophysics `KeyError` on basin defaults

Root cause: The user's simplified `libya_geology.py` was missing 8 keys that `PetrophysicsEngine` accessed (`rho_fluid`, `rsh`, `vsh_method`, `hydrostatic_gradient`, `normal_dt_surface`, `normal_dt_exp`, `typical_api`, `typical_gor`). Fix: Rewrote `libya_geology.py` with all required keys documented with units and purpose.

Bug: Log comparison stacked instead of side-by-side

Root cause: `make_comparison_plot()` used `plt.subplots(2, n_cols, sharey='row')` which creates 2 rows (one per well) and `n_cols` columns. The two wells appeared vertically stacked. Fix: Changed to `plt.subplots(1, n*2, sharey=True)` — one row of 2N tracks where the left N are Well A and right N are Well B. All tracks share the same y-axis (depth). A visual separator marker is drawn on the last Well A track.

Bug: `st.text_input` inside `st.button` block (Full Report)

Root cause: Streamlit does not allow widget creation inside a button callback block — the widget never renders because the block only executes on the rerun after button click, by which time Streamlit has already finished rendering the widget tree. Fix: Moved `project_name = st.text_input()` to ABOVE the `st.button()` call.

Bug: Temp file leak on parse error

Root cause: `parse_uploaded_las()` used `os.unlink(tmp_path)` only in the try block, so if parsing raised an exception the temp file was never deleted. Fix: Moved `os.unlink(tmp_path)` into a finally block so it always runs.

Bug: PorosityS ummary space typo

Root cause: `petro_summaries.py` had a space in the class name '`@dataclass class PorosityS ummary`' which is a Python `SyntaxError`. Fix: Rewrote `petro_summaries.py` with correct class name `PorositySummary`.

Bug: pipeline.py imports 'rich' library

Root cause: Original pipeline.py imported rich.console.Console and rich.progress.track for pretty terminal output. Rich is not in the requirements for all environments. Fix: Replaced rich with standard Python logging (logger.info()).

17. Deployment & Setup Guide

How to install and run Project_QLE

17.1 Installation

```
git clone https://github.com/Eng-qusain/Project_QLE
cd Project_QLE
pip install -r requirements.txt
# Optional: set your Gemini API key
echo 'GEMINI_API_KEY=AIZA...' > .env
# Run the application
streamlit run app.py
```

17.2 First Run — Owner Key

On first launch, the access control system creates the database and generates a master owner key. The key is printed to the terminal and saved to `~/project_qle/owner_key.txt`. Enter this key in the login screen to access the app as owner. To create user accounts, use the admin dashboard:

```
streamlit run admin_dashboard.py --server.port 8502
```

17.3 Required Environment Variables

GEMINI_API_KEY	Google Gemini API key. Can also be entered in the sidebar at runtime. Get from: https://aistudio.google.com/app/apikey
QLE_MASTER_KEY	Optional: set a specific owner key instead of auto-generating. Useful for reproducible deployments.

17.4 Key Python Dependencies

streamlit>=1.28	Minimum version for st.map() with zoom parameter.
lasio>=0.31	Minimum version for LAS 3.0 support.
google-generativeai>=0.5	For genai.list_models() which enables auto model detection.
sqlalchemy>=2.0	Version 2.0 changed several ORM APIs. Earlier versions will fail.
xgboost>=1.7	Optional — if not installed, XGBoost model is skipped gracefully.
reportlab>=3.6	Required only to regenerate this PDF documentation.

17.5 Adding a New Basin

To add a new basin (e.g. CYRENAICA), add a new entry to `_BASIN_DEFAULTS` in `core/libya_geology.py` with all 13 required keys. Then add it to `LIBYAN_BASINS` display name dictionary. The basin will automatically appear in all basin selectboxes throughout the app because they read from the `LIBYAN_BASINS` dict.

Project_QLE Technical Documentation | v1.0 | 2026 | Libya NOC | Confidential